

1 Concrete Syntax

1.1 Lexical Considerations

Keywords and identifiers are case-sensitive. For example, `if` is a keyword, but `IF` is an identifier; `foo` and `Foo` are two different identifiers referring to two distinct variables.

The keywords are: `global` `if` `else` `while` `return` `fun` `true` `false` `None`

Comments are started by `//` and are terminated by the end of the line.

White space may appear between any lexical tokens (but not within a single token). White space is defined as one or more spaces, tabs, line-break characters (carriage return, line feed, form feed), and comments.

Keywords and identifiers must be separated by white space, or a token that is neither a keyword nor an identifier. For example, `intfortrue` is a single identifier, not three distinct keywords. If a sequence begins with an alphabetic character or an underscore, then it and the longest sequence of alphanumeric characters following it forms a token.

String literals are composed of `<char>`s enclosed in double quotes.

If a sequence begins with a decimal digit, then the longest following sequence of decimal digits forms a decimal integer literal. A long sequence of digits (e.g., `123456789123456789123`) is scanned as a single token.

A `<char>` is any printable ASCII character (ASCII values between decimal value 32 and 126 inclusive) other than quote (`"`) or backslash (`\`), plus the 2-character sequences `\"` to denote quote, `\\` to denote backslash, `\t` to denote a literal tab, or `\n` to denote newline.

1.2 Reference Grammar

Notation	Meaning
<code><foo></code>	means <code>foo</code> is a nonterminal.
<code>foo</code>	(in bold font) means that <code>foo</code> is a terminal.
<code>[<i>x</i>]</code>	means zero or one occurrence of <i>x</i> , i.e., <i>x</i> is optional; note that brackets in quotes ' <code>[</code> ' ' <code>]</code> ' are terminals.
<i>x</i> *	means zero or more occurrences of <i>x</i> .
<i>x</i> ⁺ ,	a comma-separated list of one or more <i>x</i> . note that there is no comma following the last of <i>x</i> .
<code>{ }</code>	large braces are used for grouping; note that braces in quotes ' <code>{</code> ' ' <code>}</code> ' are terminals.
<code> </code>	separates alternatives.

$\langle \text{program} \rangle \rightarrow \langle \text{statement} \rangle^*$
 $\langle \text{statement} \rangle \rightarrow \begin{array}{l} \langle \text{location} \rangle = \langle \text{expr} \rangle ; \\ | \\ \langle \text{call} \rangle ; \\ | \\ \textbf{global} \langle \text{id} \rangle ; \\ | \\ \textbf{if} (\langle \text{expr} \rangle) \langle \text{block} \rangle [\textbf{else} \langle \text{block} \rangle] \\ | \\ \textbf{while} (\langle \text{expr} \rangle) \langle \text{block} \rangle \\ | \\ \textbf{return} \langle \text{expr} \rangle ; \end{array}$
 $\langle \text{block} \rangle \rightarrow \text{'{' } \langle \text{statement} \rangle^* \text{'}'}$
 $\langle \text{expr} \rangle \rightarrow \begin{array}{l} \textbf{fun} ([\langle \text{id} \rangle^+,]) \langle \text{block} \rangle \\ | \\ \text{'{' } [\langle \text{id} \rangle : \langle \text{expr} \rangle^+;] \text{'}' } \\ | \\ \langle \text{simple_expr} \rangle \end{array}$
 $\langle \text{simple_expr} \rangle \rightarrow \begin{array}{l} \langle \text{location} \rangle \\ | \\ \langle \text{call} \rangle \\ | \\ \langle \text{literal} \rangle \\ | \\ - \langle \text{simple_expr} \rangle \\ | \\ ! \langle \text{simple_expr} \rangle \\ | \\ \langle \text{simple_expr} \rangle \langle \text{bin_op} \rangle \langle \text{simple_expr} \rangle \\ | \\ (\langle \text{simple_expr} \rangle) \end{array}$
 $\langle \text{bin_op} \rangle \rightarrow \langle \text{arith_op} \rangle \mid \langle \text{comp_op} \rangle \mid \langle \text{cond_op} \rangle$
 $\langle \text{arith_op} \rangle \rightarrow + \mid - \mid * \mid /$
 $\langle \text{comp_op} \rangle \rightarrow < \mid > \mid <= \mid >= \mid ==$
 $\langle \text{cond_op} \rangle \rightarrow \& \mid \mid$
 $\langle \text{call} \rangle \rightarrow \langle \text{location} \rangle ([\langle \text{expr} \rangle^+,])$
 $\langle \text{location} \rangle \rightarrow \begin{array}{l} \langle \text{id} \rangle \\ | \\ \langle \text{location} \rangle . \langle \text{id} \rangle \\ | \\ \langle \text{location} \rangle \text{'[' } \langle \text{expr} \rangle \text{'}' } \end{array}$
 $\langle \text{literal} \rangle \rightarrow \langle \text{int_literal} \rangle \mid \langle \text{string_literal} \rangle \mid \langle \text{bool_literal} \rangle \mid \textbf{None}$
 $\langle \text{int_literal} \rangle \rightarrow \langle \text{digit} \rangle \langle \text{digit} \rangle^*$
 $\langle \text{string_literal} \rangle \rightarrow \text{" } \langle \text{char} \rangle^* \text{"}$
 $\langle \text{bool_literal} \rangle \rightarrow \textbf{true} \mid \textbf{false}$
 $\langle \text{id} \rangle \rightarrow \langle \text{alpha} \rangle \langle \text{alpha_num} \rangle^*$
 $\langle \text{alpha_num} \rangle \rightarrow \langle \text{alpha} \rangle \mid \langle \text{digit} \rangle$
 $\langle \text{alpha} \rangle \rightarrow \text{a} \mid \text{b} \mid \dots \mid \text{z} \mid \text{A} \mid \text{B} \mid \dots \mid \text{Z} \mid _$
 $\langle \text{digit} \rangle \rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9$

1.3 Operator Precedence and Associativity

Operator precedence is a set of rules that determine the priority with which operators are evaluated in expressions containing multiple operators. Specifically, operators with higher precedence are grouped and applied before those with lower precedence, allowing accurate translation of written formulas into executable behavior without requiring explicit parentheses.

Operator associativity refers to the set of rules that determine the order in which operators of the same precedence are evaluated in an expression. For binary operators, this occurs either left to right or right to left.

Operator precedence is defined in the following order from highest to lowest:

<i>Operators</i>	<i>Comments</i>
-	unary minus
* /	multiplication, division
+ -	addition, subtraction
< <= >= > ==	relational
!	conditional not
&	conditional and
	conditional or

All binary operators are left associative in MITScript.

2 Abstract Syntax

An MITScript program consists of a sequence of *statements*, which may contain *expressions* and *operators* as well as *identifiers* $x \in X$ that denote program variables or field names of records. The core syntax of the language is specified by the following abstract grammar:

Expression $e ::= x \mid \mathbf{true} \mid \mathbf{false} \mid n \mid st \mid \mathbf{None} \mid e_1 \text{ lop } e_2 \mid e_1 \text{ op } e_2 \mid e_1 \text{ cmp } e_2 \mid uop \ e$
 $\mid \{x_1 : e_1, \dots, x_n : e_n\} \mid e.x \mid e_1[e_2] \mid \mathbf{fun} \ x \ s \mid e_1(e_2)$

Statement $s ::= x = e \mid e_1.x = e_2 \mid e_1[e_2] = e_3 \mid \mathbf{if} \ e \ s_1 \ \mathbf{else} \ s_2 \mid \mathbf{while} \ e \ s \mid s_1; s_2 \mid \mathbf{global} \ x \mid \mathbf{return} \ e$

Operator $lop ::= \& \mid \mid \quad op ::= + \mid - \mid * \mid / \quad cmp ::= < \mid > \mid <= \mid >= \mid == \quad uop ::= - \mid !$

Note that this abstract syntax, which we use to specify the language semantics, deviates from the concrete syntax you will use in your implementation to parse real MITScript programs. The concrete syntax specifies details such as the delimiting of blocks via curly braces and the precedence of operators. While the abstract syntax assumes for simplicity that functions take exactly one argument, the concrete syntax generalizes them to any number of arguments. The grammar for the concrete syntax can be found in Section 1.2.

3 Semantics Preliminaries

Values. The semantics of the language is defined in terms of *values* $v \in V$, which correspond to the objects that are created, stored in memory, and bound to variables as the program evaluates:

Value $v ::= \mathbf{Bool}(b) \mid \mathbf{Integer}(n) \mid \mathbf{String}(st) \mid \mathbf{Record}(a) \mid \mathbf{Function}(a, x, s) \mid \mathbf{None}$

A value in the program falls in one of the following six categories:

- A *Boolean* is denoted by the constructor `Bool(b)`, where *b* is either `true` or `false`.
- An *integer* is denoted by the constructor `Integer(n)`, where $n \in \mathbb{Z}$.
- A *string* is denoted by the constructor `String(st)`, where *st* is an ASCII string (including standard escape sequences).
- A *record* is denoted by the constructor `Record(a)`, where $a \in A = \mathbb{N} \cup \{\cdot\}$ is an *address*, either a natural number or the distinguished value $\cdot$ which represents an invalid address, similar to a null pointer in other languages. For a record, the address *a* in question always points to a map $m \in X \rightarrow A$ from field names to addresses of field values.
- A *function* is denoted by the constructor `Function(a, x, s)`, where *a* is the address of the function's *stack frame* (to be defined in Section 3), *x* is the function's formal argument, and *s* is the statement corresponding to the function's body.
- A *null* value is denoted by the singleton value `None`.

These explicit constructors do not appear literally in the syntax of MITScript programs, which instead consists of expressions and statements. For example, rather than an instance of `Record(a)`, a program would contain an instance of the expression $\{x_1 : e_1, \dots, x_n : e_n\}$. Instead, values represent the physical objects stored in memory as the program executes, including their under-the-hood bookkeeping.

Program State. The state of an MITScript program is represented by a *heap* *h* and a *stack* γ . The heap *h* is a mapping from addresses *a* to objects allocated on the heap, each of which is either a value *v* or a *stack frame* σ . The stack γ is a sequence of addresses of stack frames allocated on the heap.

Stack Frames. A *stack frame* $\sigma \in (X \cup \{\text{parent}, \text{global}\}) \rightarrow A$ is a mapping from program variables to addresses that point to the contents of the program variables stored in the heap. A stack frame has two additional distinguished fields. The `parent` field stores the address of the stack frame's parent frame. The `global` field stores a pair (X_g, a) , where X_g is a set of program variables that have been declared as global within the stack frame by `global` statements and *a* is the address of a distinguished global stack frame.

We use the notation $\gamma = \gamma'; a$ to denote that the stack γ is equal to another stack γ' with an address *a* added at the top. Similarly, we use the notation $\gamma = a_1; \gamma'; a_2$ to denote that γ is a stack that has *a*₁ at the bottom, followed by another stack γ' followed by an address *a*₂ at the top.

Program Execution. The program starts its execution with a distinguished global stack frame $\sigma_g = \{\text{parent} : \cdot, \text{global} : (\{\}, \cdot)\}$. The initial heap h_0 contains only this initial global stack frame and a pre-allocated value for `None`, i.e. $h_0 = \{a_g : \sigma_g, a_{\text{None}} : \text{None}\}$. The initial stack γ_0 contains the address of the global stack frame, i.e. $\gamma_0 = a_g$. Additionally, it should contain any definitions that are needed to appropriately support MITScript's native functions (Section 7).

Errors. When a program performs an illegal operation, execution must stop, and the interpreter must report one of the following exceptions by printing the type of the exception to console via standard output:

- **UninitializedVariableException:** when the program attempts to read from a variable that is not present in any appropriate stack frame.
- **IllegalCastException:** when the program attempts to apply an operation to a value for which it does not apply.
- **IllegalArithmeticException:** when the program attempts to divide by zero.
- **RuntimeException:** other illegal operations, such as passing too many arguments to a function.

The semantics in the following sections indicates exactly when each exception must be reported.

Integer Representation. You should implement MITScript’s integers under the standard semantics of 32-bit two’s-complement signed integers. You do not need to report errors for integer overflow or underflow.

Native Functions. In addition to the core syntax and semantics of MITScript, your implementation must support the three native, predefined global functions described in Section 7.

Scoping. In MITScript, a program variable x may reside in different stack frames depending on whether it is global. To identify the appropriate stack frame for a newly declared variable, we define in Figure 1 a function named **lookup-write** that takes the address a_1 of the current stack frame, the current heap h , and the variable x , and returns the address of the stack frame into which x should be written. This address is either the address a_2 of the global frame if x is global, or the address a_1 of the current frame otherwise.

Similarly, we define in Figure 2 a function named **lookup-read** that determines the address of the stack frame in which an existing variable x resides. This address is either that of the global frame, the current frame, or some parent of the current frame. If the variable has not been defined in any appropriate stack frame, then the interpreter must raise an **UninitializedVariableException**.

4 Statement Semantics

Figure 3 presents the semantics of statements. The evaluation relation for statements $(\gamma, h, s) \rightarrow \eta$ denotes that execution of a statement s from a stack γ and a heap h yields a *heap-value* η . A heap value η is either a heap h or a *return-value* denoted by the constructor **Return**(h, v), where h is a heap and v is a value. The purpose of the distinguished return-value is to enable the semantics specification to appropriately handle cases where **return** statements appear arbitrarily within the body of a function.

$$\begin{array}{c}
\text{GLOBALLOOKUPWRITE} \\
\frac{h(a_1) = \sigma_1 \quad \sigma_1(\text{global}) = (X_g, a_2) \quad x \in X_g}{\text{lookup-write}(a_1, h, x) = a_2} \\
\\
\text{LOCALLOOKUPWRITE} \\
\frac{h(a_1) = \sigma_1 \quad \sigma_1(\text{global}) = (X_g, a_2) \quad x \notin X_g}{\text{lookup-write}(a_1, h, x) = a_1}
\end{array}$$

Figure 1: Semantics of the lookup-write function.

$$\begin{array}{c}
\text{GLOBALLOOKUPREAD} \\
\frac{\sigma_1(\text{global}) = (X_g, a_2) \quad x \in X_g \quad h(a_1) = \sigma_1 \quad h(a_2) = \sigma_2 \quad x \in \text{dom}(\sigma_2) \quad \sigma_2(x) = a_3}{\text{lookup-read}(a_1, h, x) = a_3} \\
\\
\text{LOCALLOOKUPREAD} \\
\frac{h(a_1) = \sigma_1 \quad \sigma_1(\text{global}) = (X_g, a_2) \quad x \notin X_g \quad x \in \text{dom}(\sigma_1) \quad \sigma_1(x) = a_3}{\text{lookup-read}(a_1, h, x) = a_3} \\
\\
\text{SCOPEDLOOKUPREAD} \\
\frac{h(a_1) = \sigma_1 \quad \sigma_1(\text{global}) = (X_g, a_2) \quad x \notin X_g \quad x \notin \text{dom}(\sigma_1) \quad \text{lookup-read}(\sigma(\text{parent}), h, x) = a_3}{\text{lookup-read}(a_1, h, x) = a_3}
\end{array}$$

Figure 2: Semantics of the lookup-read function.

VARASSIGNMENT		
$\frac{(\gamma; a_1, h, e) \rightarrow (h', v) \quad \text{lookup-write}(a_1, h', x) = a_2 \quad \sigma = h'(a_2) \quad a_3 \notin \text{dom}(h') \quad \sigma' = \sigma[x : a_3]}{(\gamma; a_1, h, x = e) \rightarrow h'[a_3 : v][a_2 : \sigma']}$		
HEAPASSIGNMENT		
$\frac{(\gamma, h, e_1) \rightarrow (h', \text{Record}(a_1)) \quad (\gamma, h', e_2) \rightarrow (h'', v) \quad h''(a_1) = m \quad a_2 \notin \text{dom}(h'') \quad m' = m[x : a_2]}{(\gamma, h, e_1.x = e_2) \rightarrow h''[a_2 : v][a_1 : m']}$		
HEAPINDEXASSIGNMENT		
$\frac{x = \text{str}(v_1) \quad (\gamma, h, e_1) \rightarrow (h', \text{Record}(a_1)) \quad (\gamma, h', e_2) \rightarrow (h'', v_1) \quad (\gamma, h'', e_3) \rightarrow (h''', v_2) \quad h'''(a_1) = m \quad a_2 \notin \text{dom}(h''') \quad m' = m[x : a_2]}{(\gamma, h, e_1[e_2] = e_3) \rightarrow h'''[a_2 : v_2][a_1 : m']}$		
IFTRUE		
$\frac{(\gamma, h, e) \rightarrow (h', \text{Bool}(\text{true})) \quad (\gamma, h', s_1) \rightarrow \eta}{(\gamma, h, \text{if } e \text{ } s_1 \text{ else } s_2) \rightarrow \eta}$		
IFFALSE		
$\frac{(\gamma, h, e) \rightarrow (h', \text{Bool}(\text{false})) \quad (\gamma, h', s_2) \rightarrow \eta}{(\gamma, h, \text{if } e \text{ } s_1 \text{ else } s_2) \rightarrow \eta}$		
WHILE		
$\frac{(\gamma, h, \text{if } e \text{ } (s; \text{while } e \text{ } s)) \rightarrow \eta}{(\gamma, h, \text{while } e \text{ } s) \rightarrow \eta}$		
SEQUENCE		
$\frac{(\gamma, h, s_1) \rightarrow h' \quad (\gamma, h', s_2) \rightarrow \eta}{(\gamma, h, s_1; s_2) \rightarrow \eta}$		
SEQUENCEReturn		
$\frac{(\gamma, h, s_1) \rightarrow \text{Return}(h', v)}{(\gamma, h, s_1; s_2) \rightarrow \text{Return}(h', v)}$		
GLOBAL		
$\frac{}{(\gamma, h, \text{global } x) \rightarrow h}$		
RETURN		
$\frac{(\gamma, h, e) \rightarrow (h', v)}{(\gamma, h, \text{return } e) \rightarrow \text{Return}(h', v)}$		

Figure 3: Semantics of statements.

Variable Assignment. This rule first evaluates the expression e to a value v and a new heap h' . First, it uses lookup-write (Section 3) to determine the address a_2 of the stack frame σ in which to update the variable x , using the appropriate scoping rules. It allocates a new address a_3 previously unused in h' . It returns a new stack with x set to a_3 and a new heap with a_3 pointing to v and a_2 pointing to the new stack.

Heap Assignment. This rule does not update a stack frame but rather updates a record in the heap. This statement must raise an `IllegalCastException` if the result of the evaluation of e_1 is not a record.

Heap Index Assignment. This rule is similar to the previous one, except the field name is dynamic. The index expression is evaluated and cast to a string using the `str` function, which is defined in Section 6. The operation must raise an `IllegalCastException` if the result of evaluation of e_1 is not a record.

If. These rules execute the first branch if the condition evaluates to true and the second branch otherwise. The statement must raise an `IllegalCastException` if the result of the evaluation of e is not a Boolean.

While. This rule repeatedly executes its body while the condition evaluates to true. The statement must raise an `IllegalCastException` if the result of the evaluation of e is not a Boolean.

Sequence. This rule evaluates the first statement and then evaluates the second statement. Note that if the first statement yields a return-value, then the program skips the execution of the second statement.

Global. This rule is a no-op; its semantics is only incorporated later during function calls.

Return. This rule evaluates the expression and yields a return-value.

5 Expression Semantics

Figures 4 to 6 present the semantics of expressions. The evaluation relation for expressions $(\gamma, h, e) \rightarrow (h', v)$ denotes that given a stack γ and a heap h , an expression e evaluates to a new heap h' and a value v .

Constants. For each type of constant value, the rule generates a value corresponding to the constant.

Logical Operators. Logical operators can only be applied to Boolean values.

$\frac{\text{INTEGERCONSTANT}}{(\gamma, h, n) \rightarrow (h, \text{Integer}(n))}$		$\frac{\text{BOOLEANCONSTANTTRUE}}{(\gamma, h, \text{true}) \rightarrow (h, \text{Bool}(\text{true}))}$	
$\frac{\text{BOOLEANCONSTANTFALSE}}{(\gamma, h, \text{false}) \rightarrow (h, \text{Bool}(\text{false}))}$	$\frac{\text{STRINGCONSTANT}}{(\gamma, h, st) \rightarrow (h, \text{String}(st))}$	$\frac{\text{NONECONSTANT}}{(\gamma, h, \text{None}) \rightarrow (h, \text{None})}$	

Figure 4: Semantics of constant expressions.

Arithmetic Operators. The arithmetic operators `-`, `*`, and `/` can only be applied to integer values. The interpreter must generate an `IllegalArithmeticException` for division by zero.

The operator `+` on two integers corresponds to standard integer addition. The operator `+` on two strings corresponds to string concatenation. The operator `+` on a string and a value that is not a string (where either can appear on either side of the `+`) causes the non-string value to be cast to a string.

Unary Operators. The unary minus operator `-` can only be applied to an integer value. The unary negation operator `!` can only be applied to a Boolean value.

Comparison Operators. The comparison operators `<`, `>`, `<=`, `>=` are only defined for integers.

Two integers, two Booleans, or two strings are equal if their values are equal. The value `None` is equal to itself. Equality between two records checks that their addresses are identical. Equality between two functions checks that their stack frame addresses are the same, their ordered list of named arguments are the same, and their bodies are the same ($s_1 = s_2$ checks the ASTs are the same, so make sure you have no redundant nodes). Equality between two values of different type returns `false`.

Operator Errors. For the above operators, the interpreter must generate an `IllegalCastException` for any other operation that is not given an explicit semantics by the rules.

Variable Read. This rule specifies the semantics of reading from a variable, using `lookup-read` (Section 3) to obtain the appropriate address of x .

Record Constructor. This rule evaluates all of the expressions inside the list and constructs a `Record` value from fields to values. Note that the initializer expressions must be evaluated in order.

Field Read. These rules evaluate the expression to a `Record` value, from which the field x is looked up. If the record does not contain the named field x , the expression evaluates to `None`. The statement raises an `IllegalCastException` if the result of the evaluation of e is not a record.

$$\begin{array}{c}
\text{LOGICALOPERATION} \\
\frac{(\gamma, h, e_1) \rightarrow (h', \text{Bool}(b_1)) \quad (\gamma, h', e_2) \rightarrow (h'', \text{Bool}(b_2))}{(\gamma, h, e_1 \text{ lop } e_2) \rightarrow (h'', \text{Bool}(b_1 \text{ lop } b_2))} \\
\\
\text{ARITHMETICOPERATION} \\
\frac{(\gamma, h, e_1) \rightarrow (h', \text{Integer}(n_1)) \quad (\gamma, h', e_2) \rightarrow (h'', \text{Integer}(n_2))}{(\gamma, h, e_1 \text{ op } e_2) \rightarrow (h'', \text{Integer}(n_1 \text{ op } n_2))} \\
\\
\begin{array}{cc}
\text{UNARYMINUS} & \text{UNARYNOT} \\
\frac{(\gamma, h, e) \rightarrow (h', \text{Integer}(n))}{(\gamma, h, -e) \rightarrow (h', \text{Integer}(-n))} & \frac{(\gamma, h, e) \rightarrow (h', \text{Bool}(b))}{(\gamma, h, !e) \rightarrow (h', \text{Bool}(\neg b))}
\end{array} \\
\\
\text{COMPARISONOPERATION} \\
\frac{(\gamma, h, e_1) \rightarrow (h', \text{Integer}(n_1)) \quad (\gamma, h', e_2) \rightarrow (h'', \text{Integer}(n_2))}{(\gamma, h, e_1 \text{ cmp } e_2) \rightarrow (h'', \text{Bool}(n_1 \text{ cmp } n_2))} \\
\\
\text{STRINGCONCATENATION} \\
\frac{(\gamma, h, e_1) \rightarrow (h', \text{String}(st_1)) \quad (\gamma, h', e_2) \rightarrow (h'', \text{String}(st_2))}{(\gamma, h, e_1 + e_2) \rightarrow (h'', \text{String}(st_1 + st_2))} \\
\\
\text{STRINGCONCATENATIONLEFTCAST} \\
\frac{(\gamma, h, e_1) \rightarrow (h', t(\delta)) \quad (\gamma, h', e_2) \rightarrow (h'', \text{String}(st_2)) \quad t \neq \text{String} \quad st_1 = \text{str}(t(\delta))}{(\gamma, h, e_1 + e_2) \rightarrow (h'', \text{String}(st_1 + st_2))} \\
\\
\text{STRINGCONCATENATIONRIGHTCAST} \\
\frac{(\gamma, h, e_1) \rightarrow (h', \text{String}(st_1)) \quad (\gamma, h', e_2) \rightarrow (h'', t(\delta)) \quad t \neq \text{String} \quad st_2 = \text{str}(t(\delta))}{(\gamma, h, e_1 + e_2) \rightarrow (h'', \text{String}(st_1 + st_2))} \\
\\
\text{NONEEQUALITY} \\
\frac{(\gamma, h, e_1) \rightarrow (h', \text{None}) \quad (\gamma, h', e_2) \rightarrow (h'', \text{None})}{(\gamma, h, e_1 == e_2) \rightarrow (h'', \text{Bool}(\text{true}))} \\
\\
\text{PRIMITIVEEQUALITY} \\
\frac{(\gamma, h, e_1) \rightarrow (h', t_1(\delta_1)) \quad (\gamma, h', e_2) \rightarrow (h'', t_2(\delta_2)) \quad t_1 = t_2}{(\gamma, h, e_1 == e_2) \rightarrow (h'', \text{Bool}(\delta_1 = \delta_2))} \quad t_1, t_2 \in \{\text{Integer}, \text{Bool}, \text{String}\} \\
\\
\text{RECORDEQUALITY} \\
\frac{(\gamma, h, e_1) \rightarrow (h', \text{Record}(a_1)) \quad (\gamma, h', e_2) \rightarrow (h'', \text{Record}(a_2))}{(\gamma, h, e_1 == e_2) \rightarrow (h'', \text{Bool}(a_1 = a_2))} \\
\\
\text{FUNCTIONEQUALITYTRUE} \\
\frac{(\gamma, h, e_1) \rightarrow (h', \text{Function}(a_1, x, s_1)) \quad (\gamma, h', e_2) \rightarrow (h'', \text{Function}(a_2, x, s_2))}{(\gamma, h, e_1 == e_2) \rightarrow (h'', \text{Bool}(a_1 = a_2 \wedge x_1 = x_2 \wedge s_1 = s_2))} \\
\\
\text{PRIMITIVEEQUALITYMISMATCHED} \\
\frac{(\gamma, h, e_1) \rightarrow (h', t_1(\delta_1)) \quad (\gamma, h', e_2) \rightarrow (h'', t_2(\delta_2)) \quad t_1 \neq t_2}{(\gamma, h, e_1 == e_2) \rightarrow (h'', \text{Bool}(\text{false}))}
\end{array}$$

Figure 5: Semantics of arithmetic and logical expressions.

Index Read. These rules evaluate the dynamic index value. If it evaluates to a string, the field by that name is obtained from the record. If it evaluates to some other value, then that value is first cast to a string (Section 6). As before, if there is no such field present in the record, then the expression evaluates to **None**.

Function Creation. This rule creates a new function value that captures the current frame.

Function Call. There are multiple steps to this rule. First, the base expression for the function call is evaluated to a function value, which consists of the address of a frame and the code for the function.

The next step is to allocate a new stack frame for the function call. The rule creates a new stack frame whose parent is the function’s stack frame. The rule traverses the body of the function, and for every global declaration `global x`, it adds x to the set of global variables for the new stack frame. A helper function `globals` (Figure 7) performs this traversal. Then, for every assignment of the form $x = e$ in the body of f , the rule adds variable x to the stack frame and initializes it to **None**. A helper function `assigns` (Figure 8) performs this traversal. Finally, the rule evaluates the body of the function with the new stack and heap.

Note that the rule is written assuming a function with one argument, and should be generalized to support multiple arguments. When there are multiple arguments, they must be evaluated in order from left to right. If e_1 does not evaluate to a **Function** value, then `IllegalCastException` must be raised. If the caller passes too many or too few arguments to the function, then `RuntimeException` must be raised.

Return. These rules specify that a function call evaluates to the return-value if it exists or else to **None**.

6 Cast Semantics

The semantics of MITScript requires that certain values be automatically converted into strings. This conversion is performed via a function called `str`, defined by the following rules:

- $\text{str}(\text{String}(st)) = st$
- $\text{str}(\text{Boolean}(b)) = \text{“true”}$ if b is true, or **“false”** otherwise
- $\text{str}(\text{Integer}(n)) =$ base-10 representation of n , with no leading zeroes, preceded by sign if negative
- $\text{str}(\text{Function}(a, x, s)) = \text{“FUNCTION”}$
- $\text{str}(\{x_i : v_i, \dots\}) = \text{“\{”} + x_i + \text{“:”} + \text{str}(v_i) + \text{“ ”} + \dots + \text{“\}”}$
 - Fields should be ordered lexicographically in the resulting string (i.e., $i < j \implies x_i < x_j$)
- $\text{str}(\text{None}) = \text{“None”}$

VARIABLEREAD $\frac{\text{lookup-read}(a_1, h, x) = a_2 \quad h(a_2) = v}{(\gamma; a_1, h, x) \rightarrow (h, v)}$	
RECORD $\frac{(\gamma, h, e_1) \rightarrow (h_1, v_1) \quad \dots \quad (\gamma, h_{n-1}, e_n) \rightarrow (h_n, v_n) \quad \{a_1, \dots, a_{n+1}\} \notin \text{dom}(h_n) \quad m = \{x_1 : a_1, \dots, x_n : a_n\} \quad h' = h_n[a_1 : v_1, \dots, a_n : v_1, a_{n+1} : m]}{(\gamma, h, \{x_1 : e_1, \dots, x_n : e_n\}) \rightarrow (h', \text{Record}(a_{n+1}))}$	
FIELDREAD $\frac{(\gamma, h, e) \rightarrow (h', \text{Record}(a_1)) \quad h'(a_1) = m \quad x \in \text{dom}(m) \quad a_2 = m(x)}{(\gamma, h, e.x) \rightarrow (h', h'(a_2))}$	
FIELDREADFAIL $\frac{(\gamma, h, e) \rightarrow (h', \text{Record}(a)) \quad h'(a) = m \quad x \notin \text{dom}(m)}{(\gamma, h, e.x) \rightarrow (h', \text{None})}$	
INDEXREAD $\frac{(\gamma, h, e_1) \rightarrow (h', \text{Record}(a_1)) \quad h'(a_1) = m \quad (\gamma, h', e_2) \rightarrow (h'', v_1) \quad x = \text{str}(v_1) \quad x \in \text{dom}(m) \quad a_2 = m(x)}{(\gamma, h, e_1[e_2]) \rightarrow (h'', h'(a_2))}$	
INDEXREADFAIL $\frac{(\gamma, h, e_1) \rightarrow (h', \text{Record}(a)) \quad h'(a) = m \quad (\gamma, h', e_2) \rightarrow (h'', v_1) \quad x = \text{str}(v_1) \quad x \notin \text{dom}(m)}{(\gamma, h, e_1[e_2]) \rightarrow (h'', \text{None})}$	
FUNCTION $\frac{}{(\gamma; a, h, \text{fun } x \text{ } s) \rightarrow (h, \text{Function}(a, x, s))}$	
FUNCTIONCALL $\frac{(a_g; \gamma; a, h_1, e_1) \rightarrow (h_2, \text{Function}(a_c, x, s)) \quad (a_g; \gamma; a, h_2, e_2) \rightarrow (h_3, v) \quad a_2 \notin \text{dom}(h_3) \quad a_3 \notin (\text{dom}(h_3) \cup \{a_2\}) \quad \sigma = \{\text{parent} : a_c, \text{global} : (\text{globals}(s), a_g)\} \{x_i : a_{\text{None}} \mid x_i \in \text{assigns}(s)\} [x : a_2] \quad h_4 = h_3[a_2 : v][a_3 : \sigma] \quad (a_g; \gamma; a_3, h_4, s) \rightarrow \eta}{(a_g; \gamma; a, h_1, e_1(e_2)) \rightarrow \eta}$	
$\text{FUNCTIONCALLRETURN}$ $\frac{(\gamma, h, e_1(e_2)) \rightarrow \text{Return}(h', v)}{(\gamma, h, e_1(e_2)) \rightarrow (h', v)}$	$\text{FUNCTIONCALLNORETURN}$ $\frac{(\gamma, h, e_1(e_2)) \rightarrow h'}{(\gamma, h, e_1(e_2)) \rightarrow (h', \text{None})}$

Figure 6: Semantics of data and function expressions.

ASSIGN	HEAPASSIGNMENT	HEAPINDEXASSIGNMENT
$\overline{\text{globals}(x = e) = \{\}}$	$\overline{\text{globals}(e_1.x = e_2) = \{\}}$	$\overline{\text{globals}(e_1[e_2] = e_3) = \{\}}$
IF		WHILE
$\overline{\text{globals}(\text{if } e \text{ } s_1 \text{ else } s_2) = \text{globals}(s_1) \cup \text{globals}(s_2)}$		$\overline{\text{globals}(\text{while } e \text{ } s) = \text{globals}(s)}$
SEQUENCE		GLOBAL
$\overline{\text{globals}(s_1; s_2) = \text{globals}(s_1) \cup \text{globals}(s_2)}$		$\overline{\text{globals}(\text{global } x) = \{x\}}$
	RETURN	
	$\overline{\text{globals}(\text{return } e) = \{\}}$	

Figure 7: Definition of the `globals` function.

ASSIGN	HEAPASSIGNMENT	HEAPINDEXASSIGNMENT
$\overline{\text{assigns}(x = e) = \{x\}}$	$\overline{\text{assigns}(e_1.x = e_2) = \{\}}$	$\overline{\text{assigns}(e_1[e_2] = e_3) = \{\}}$
IF		WHILE
$\overline{\text{assigns}(\text{if } e \text{ } s_1 \text{ else } s_2) = \text{assigns}(s_1) \cup \text{assigns}(s_2)}$		$\overline{\text{assigns}(\text{while } e \text{ } s) = \text{assigns}(s)}$
SEQUENCE		GLOBAL
$\overline{\text{assigns}(s_1; s_2) = \text{assigns}(s_1) \cup \text{assigns}(s_2)}$		$\overline{\text{assigns}(\text{global } x) = \{\}}$
	RETURN	
	$\overline{\text{assigns}(\text{return } e) = \{\}}$	

Figure 8: Definition of the `assigns` function.

7 Native Functions

In addition to the core syntax and semantics of MITScript, your interpreter must support the following three native functions:

- `print(s)` Cast `s` to a string and print it to the console (via standard output) followed by a newline.
- `input()` Read a line of input from the console (via standard input) and return it as a string value.
- `intcast(s)` Parse `s` as an integer value. If the string does not represent an integer (e.g., “hello”), this function raises an `IllegalCastException`. You may internally utilize the C function `atoi`.

These functions should be initially available in the global scope. However, it should be possible for the program to redefine them. For example, the program below updates `print` to add a prefix to every message:

```
print("Hello"); // this should print 'Hello' to the console.
oldprint = print;
print = fun(s){
    oldprint("OUTPUT: " + s);
};
print("Hello"); // this should print 'OUTPUT: Hello' to the console.
```